

Customer Shopping Behavior Analysis

End-to-End Data Analytics Project — Python | MySQL | Power BI

1. Project Overview

This project analyzes consumer shopping behavior using a transactional dataset of 3,900 purchases spanning multiple product categories. The objective is to explore spending patterns, customer segments, product preferences, and subscription behavior in order to support data-driven business decisions around marketing, loyalty, and product strategy.

The end-to-end workflow follows three stages: data cleaning and feature engineering in Python, structured business-question analysis in SQL, and visualization of key insights through an interactive Power BI dashboard.

2. Business Problem

A retail company wants to better understand its customers' shopping behavior to improve sales, customer satisfaction, and long-term loyalty. Management has observed shifts in purchasing patterns across demographics, product categories, and channels, and wants to know which factors — discounts, reviews, seasonality, or payment preferences — drive purchase decisions and repeat business.

Guiding question:

"How can the company use its consumer shopping data to identify trends, improve customer engagement, and optimize marketing and product strategy?"

3. Dataset Summary

Attribute	Detail
Rows	3,900
Columns	18 (after cleaning: 18, after dropping <code>promo_code_used</code>)
Demographics	Age, Gender, Location, Subscription Status
Purchase details	Item Purchased, Category, Purchase Amount, Season, Size, Color
Behavioral fields	Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type
Missing data	37 null values in Review Rating

4. Data Preparation (Python)

The raw CSV was loaded and cleaned using pandas before being pushed into a MySQL database for SQL analysis.

4.1 Initial Exploration

- Loaded the dataset with pandas and inspected structure using `df.info()` and `df.describe(include='all')`.
- Checked for missing values with `df.isnull().sum()` — found 37 nulls in `review_rating`.

```

<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   str
3   Item Purchased                       3900 non-null   str
4   Category                             3900 non-null   str
5   Purchase Amount (USD)                3900 non-null   int64
6   Location                             3900 non-null   str
7   Size                                 3900 non-null   str
8   Color                                3900 non-null   str
9   Season                               3900 non-null   str
10  Review Rating                        3863 non-null   float64
11  Subscription Status                  3900 non-null   str
12  Shipping Type                        3900 non-null   str
13  Discount Applied                     3900 non-null   str
14  Promo Code Used                      3900 non-null   str
15  Previous Purchases                   3900 non-null   int64
16  Payment Method                       3900 non-null   str
17  Frequency of Purchases                3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB

```

4.2 Missing Data Handling

Missing Review Rating values were imputed using the median rating within each product category, rather than a single global median, to preserve category-level rating patterns:

```

df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x:
x.fillna(x.median()))

```

4.3 Column Standardization

Column headers were converted to snake_case for consistency and easier SQL querying:

```

df.columns = df.columns.str.lower().str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})

```

4.4 Feature Engineering

- `age_group` — created by binning age into quartiles (Young Adult, Adult, Middle-aged, Senior) using `pd.qcut()`.
- `purchase_frequency_days` — mapped the `frequency_of_purchases` text values (Weekly, Fortnightly, Monthly, Quarterly, Annually, etc.) to an approximate numeric day count for trend analysis.

4.5 Redundancy Check

`discount_applied` and `promo_code_used` were found to be identical across every row, so `promo_code_used` was dropped to avoid duplicate signal:

```

df = df.drop('promo_code_used', axis=1)

```

4.6 Loading into MySQL

Unlike the original walkthrough (which used PostgreSQL), this project loads the cleaned DataFrame into a local MySQL instance using SQLAlchemy:

```

from sqlalchemy import create_engine
from sqlalchemy.engine import URL

```

```

connection_url = URL.create(
    "mysql+mysqlconnector",
    username="root",
    password="*****",
    host="localhost",
    port=3306,
    database="customer_behavior"
)

engine = create_engine(connection_url)
df.to_sql("customer", engine, if_exists="replace", index=False)

```

5. Data Analysis (SQL)

With the cleaned data loaded into MySQL, ten business questions were answered using structured queries covering revenue, customer segments, discounts, product performance, and loyalty.

Q1. Total revenue by gender

```

SELECT gender, SUM(purchase_amount) AS revenue
FROM customer
GROUP BY gender;

```

	gender	revenue
▶	Male	157890
	Female	75191

Q2. High-spending discount users

Customers who used a discount but still spent above the overall average purchase amount:

```

SELECT customer_id, purchase_amount
FROM customer
WHERE discount_applied = 'Yes'
AND purchase_amount >= (SELECT AVG(purchase_amount) FROM customer);

```

	customer_id	purchase_amount
▶	2	64
	3	73
	4	90
	7	85
	9	97
	12	68
	13	72
	16	81
	20	90
	22	62
	--	--

Q3. Top 5 products by average review rating

```
SELECT item_purchased, ROUND(AVG(review_rating),2) AS Average_review_rating
FROM customer
GROUP BY item_purchased
ORDER BY Average_review_rating DESC
LIMIT 5;
```

	item_purchased	Average_review_rating
▶	Gloves	3.86
	Sandals	3.84
	Boots	3.82
	Hat	3.8
	Handbag	3.78

Q4. Standard vs Express shipping — average purchase amount

```
SELECT shipping_type, AVG(purchase_amount) AS avg_amount
FROM customer
WHERE shipping_type IN ('Standard','Express')
GROUP BY shipping_type;
```

	shipping_type	avg_purhcase
▶	Express	60.4752
	Standard	58.4602

Q5. Subscribers vs non-subscribers

```
SELECT subscription_status, COUNT(customer_id) AS total_customers, subscription_status,
      AVG(purchase_amount) AS avg_spend,
      SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue DESC;
```

	subscription_status	total_customers	subscription_status	avg_spend	total_revenue
▶	Yes	1053	Yes	59.4919	62645
	No	2847	No	59.8651	170436

Interestingly, non-subscribers generate far higher total revenue (larger base), while average spend per customer is comparable between the two groups — subscription status alone doesn't strongly predict higher individual spend.

Q6. Products most dependent on discounts

```
SELECT item_purchased,
      ROUND(SUM(CASE WHEN discount_applied='Yes' THEN 1 ELSE 0 END) *100 / COUNT(*), 2) AS
discount_percentage
FROM customer
GROUP BY item_purchased
ORDER BY discount_percentage DESC
LIMIT 5;
```

	item_purchased	discount_percentage
▶	Hat	50.00
	Sneakers	49.66
	Coat	49.07
	Sweater	48.17
	Pants	47.37

Q7. Customer segmentation (New / Returning / Loyal)

```

WITH customer_type AS (
SELECT customer_id, previous_purchases,
CASE
WHEN previous_purchases = 1 THEN 'New'
WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
ELSE 'Loyal'
END AS customer_segment
FROM customer
)
SELECT customer_segment, COUNT(*) AS "Total Customers"
FROM customer_type
GROUP BY customer_segment;

```

	customer_segment	Total Customers
▶	Loyal	3116
	Returning	701
	New	83

Q8. Top 3 products per category

```

SELECT category, item_purchased, total_orders, item_rank FROM (
    SELECT category, item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS
item_rank
    FROM customer
    GROUP BY category, item_purchased
) ranked
WHERE item_rank <= 3;

```

	category	item_purchased	total_orders	item_rank
▶	Accessories	Jewelry	171	1
	Accessories	Sunglasses	161	2
	Accessories	Belt	161	3
	Clothing	Blouse	171	1
	Clothing	Pants	171	2
	Clothing	Shirt	169	3
	Footwear	Sandals	160	1
	Footwear	Shoes	150	2
	Footwear	Sneakers	145	3
	Outerwear	Jacket	163	1
	Outerwear	Coat	161	2

Q9. Repeat buyers and subscription likelihood

```
SELECT subscription_status, COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

	subscription_status	repeat_buyers
►	Yes	958
	No	2518

Q10. Revenue contribution by age group

```
SELECT age_group, SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY age_group
ORDER BY total_revenue DESC;
```

	age_group	total_revenue
►	Young Adult	62143
	Middle-aged	59197
	Adult	55978
	Senior	55763

6. Dashboard (Power BI)

The final insights were consolidated into an interactive Power BI dashboard using a dark navy and teal color theme, designed for better contrast and a more modern, professional look.

The dashboard includes:

- Three KPI cards — Count of Customers, Average Purchase Amount, Average Review Rating
- A donut chart — % of Customers by Subscription Status
- Two category-level bar charts — Revenue by Category, Sales by Category
- Two age-group bar charts — Revenue by Age Group, Sales by Age Group
- A filter panel — Subscription Status, Gender, Category, and Shipping Type slicers



7. Business Recommendations

- Boost subscriptions — promote exclusive perks, since subscribers currently represent a minority of the customer base despite comparable average spend.
- Strengthen loyalty programs — the majority of customers already fall into the "Loyal" segment; targeted rewards can protect this base from churn.
- Review discount strategy — a handful of products rely heavily on discounts to sell; assess whether this is eroding margin unnecessarily.
- Double down on top-rated and top-selling products in each category for merchandising and campaigns.
- Target marketing by age group and shipping preference — Young Adults and Express-shipping users contribute disproportionately to revenue.

8. Tools & Tech Stack

Stage	Tool
Data Cleaning & Feature Engineering	Python (pandas)
Database	MySQL (via SQLAlchemy + mysql-connector-python)
Analysis	SQL (MySQL)
Visualization	Power BI

9. Repository Structure

```
customer-shopping-behavior-analysis/  
├── data/  
│   └── customer_shopping_behavior.csv  
├── notebooks/  
│   └── data_cleaning_and_eda.ipynb  
├── sql/  
│   └── business_questions.sql  
├── dashboard/  
│   └── customer_behavior_dashboard.pbix  
├── report/  
│   └── project_report.pdf  
└── README.md
```